

NPTEL Course on "Computer System Optimization for ML"

Lecture #24

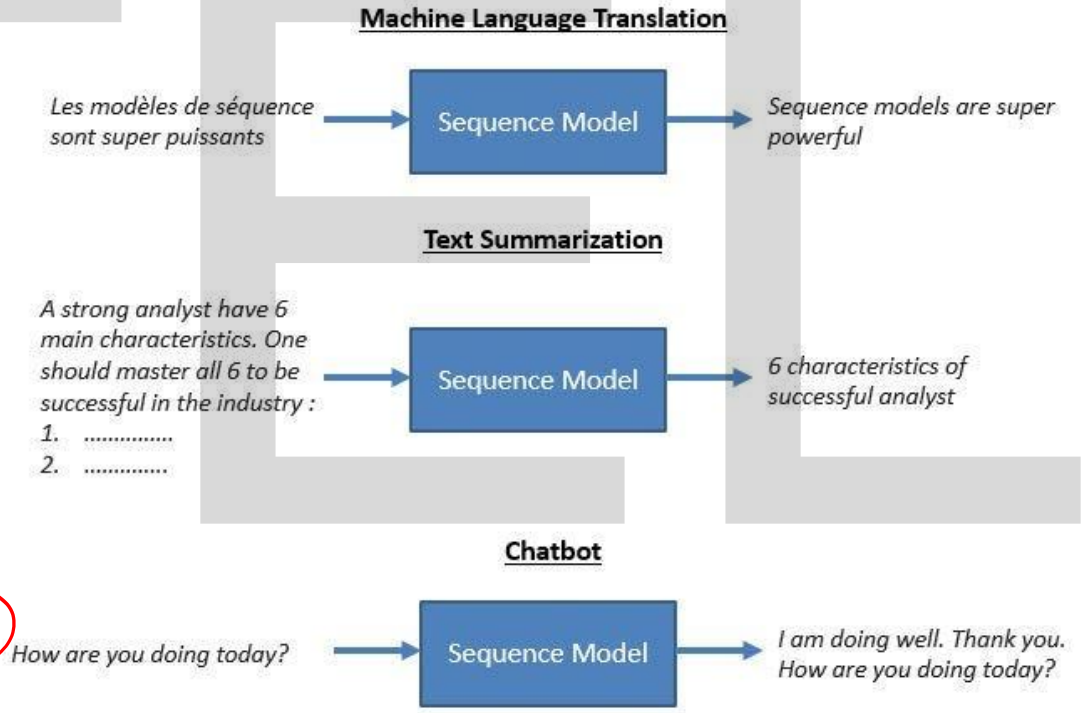
Agenda

- **Background on Large Language Models (LLMs)**

NPTEL

Sequence-to-Sequence (Seq2Seq) Task

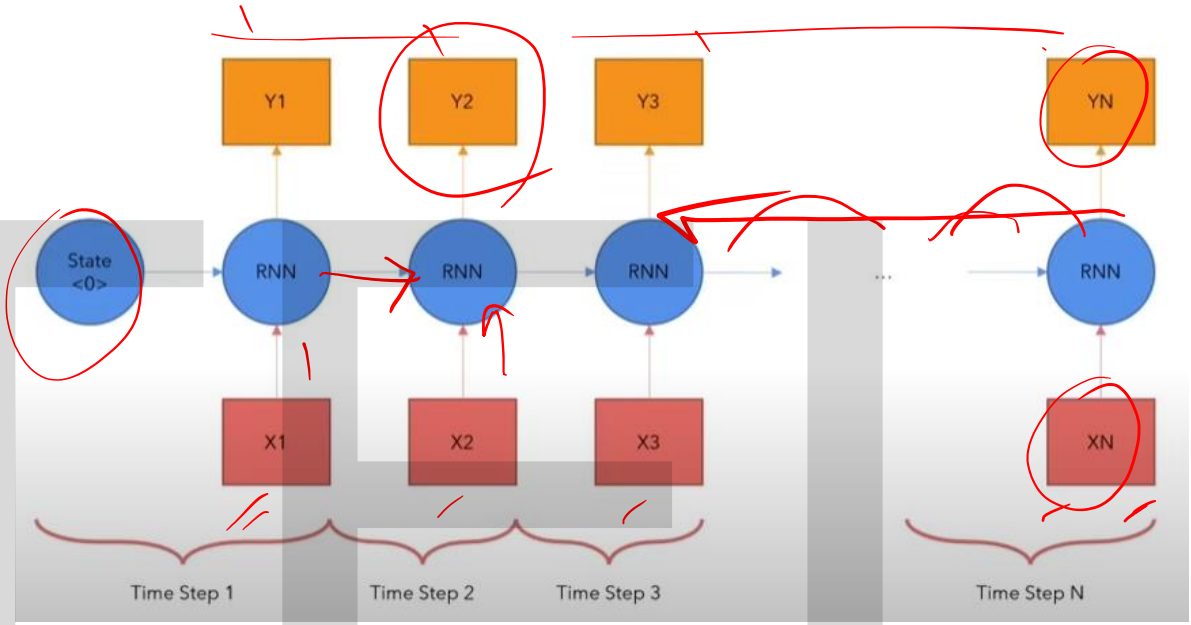
- Typically used in Natural Language Processing (NLP)
- Both inputs and outputs are a sequence of words
  - Machine language translation
  - Text summarization
  - ChatBot
- ML algorithms for Seq2Seq tasks
  - Recurrent Neural Network (RNN)
  - Transformer Algorithms
  - Transformers with Multi-head Attentions
  - ....



NPTEL

Recurrent Neural Networks (RNN)

- A sequence of input is mapped to a sequence of output
  - X is the input
  - Y is the output
- At first time step, Y1 is produced from X1 and initial state
  - Hidden state is also produced
  - Used to produce Y2
  - And so on
- N tokens will take N steps for both training and inference



NPTEL

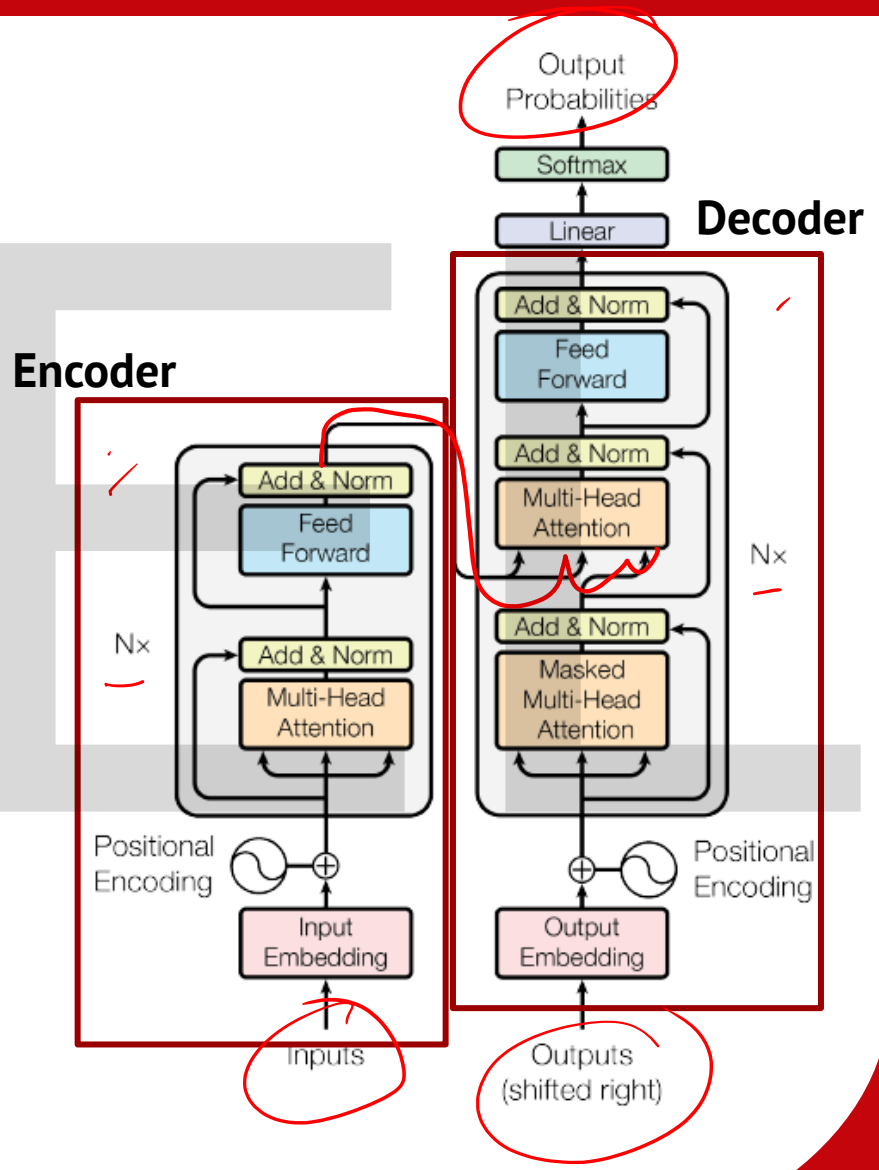
Problems with RNNs

- Each output is produced at each time step; slow
  - The longer the sequence the longer computation
- Vanishing gradient
  - For long sequences
  - Slow convergence during training
- Difficulty in accessing information from long time ago
  - Last token has very less effect from the first token
  - Typically, the context remains same across a long sequence

NPTEL

Attention is All you Need [1]

- **Based on Encoders and Decoders**
  - **They are connected**
- **Input to Encoder is a sequence of words (sentence)**
- **Input to Decoder**
  - **Outputs (shifted right)**
  - **Output probabilities**
  - **Input from Encoder**

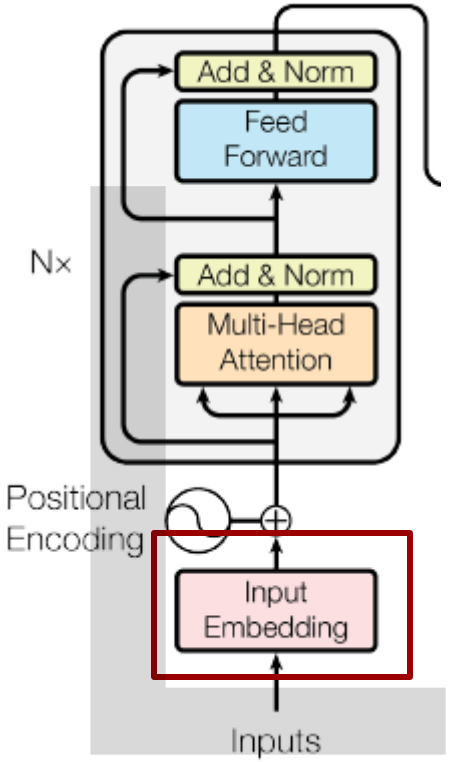


[1] Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

NDTF

Encoder: Input Embedding

- **Input Embedding**
- **Tokenizing a sentence**
  - **Not necessarily into words**
- **Assign ID to each token (from a dictionary)**
  - **Same token will have same ID**



Original sentence (tokens)

Input IDs (position in the vocabulary)

YOUR	CAT	IS	A	LOVELY	CAT
105	6587	5475	3578	65	6587

Encoder: Input Embedding

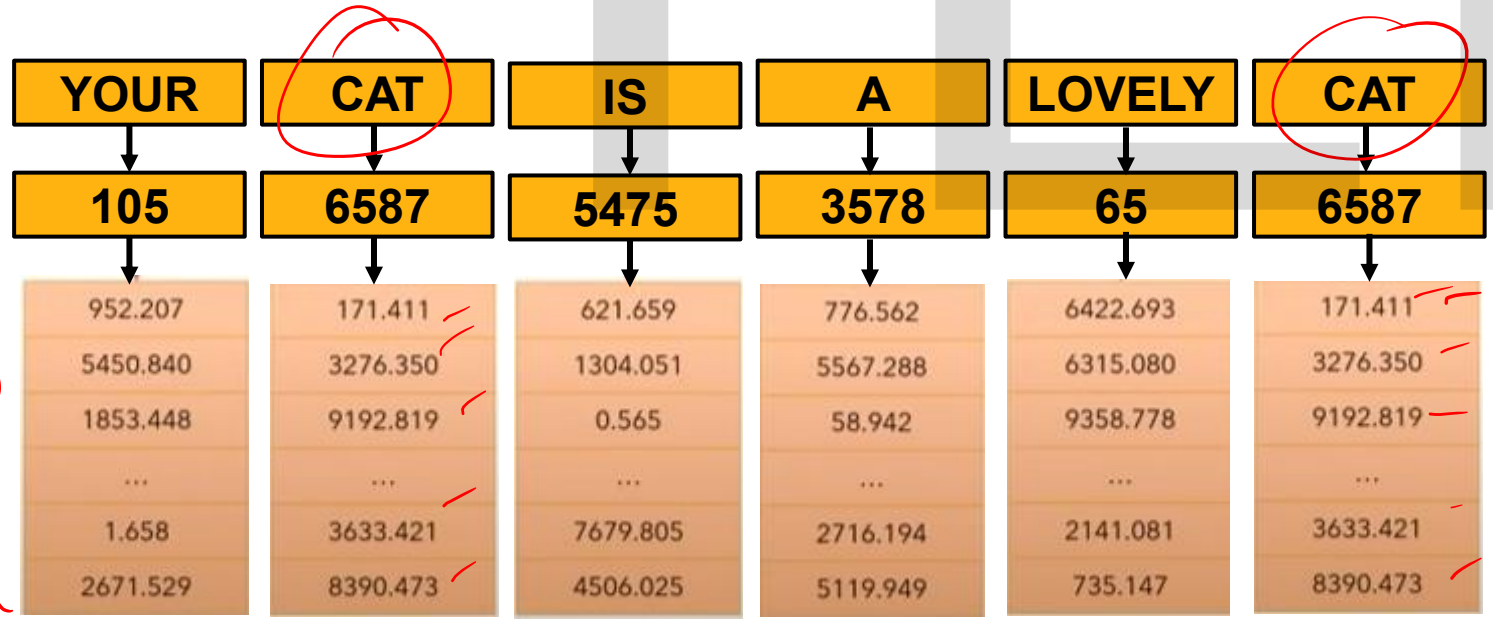
- **Input Embedding**
- **Same word is mapped to same vector**
- **The numbers are not fixed**
  - **Represent the meaning of the word**
  - **Learnable parameters**

Original sentence (tokens)

Input IDs (position in the vocabulary)

Embedding (vector of size 512)

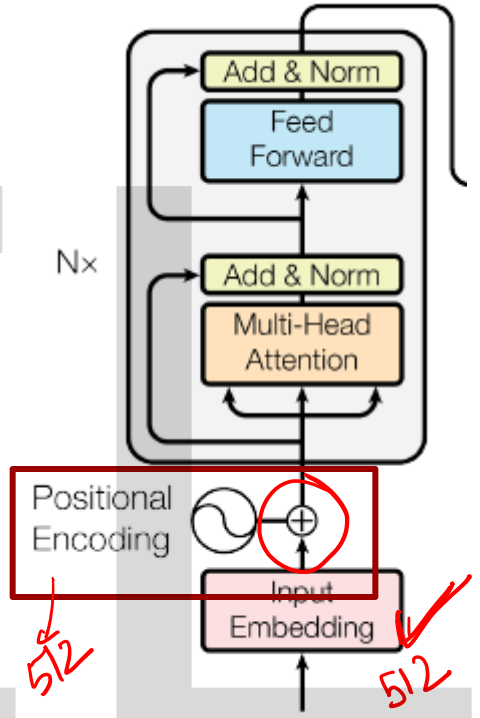
d_{model}=512



NPTEL

Encoder: Positional Encoding

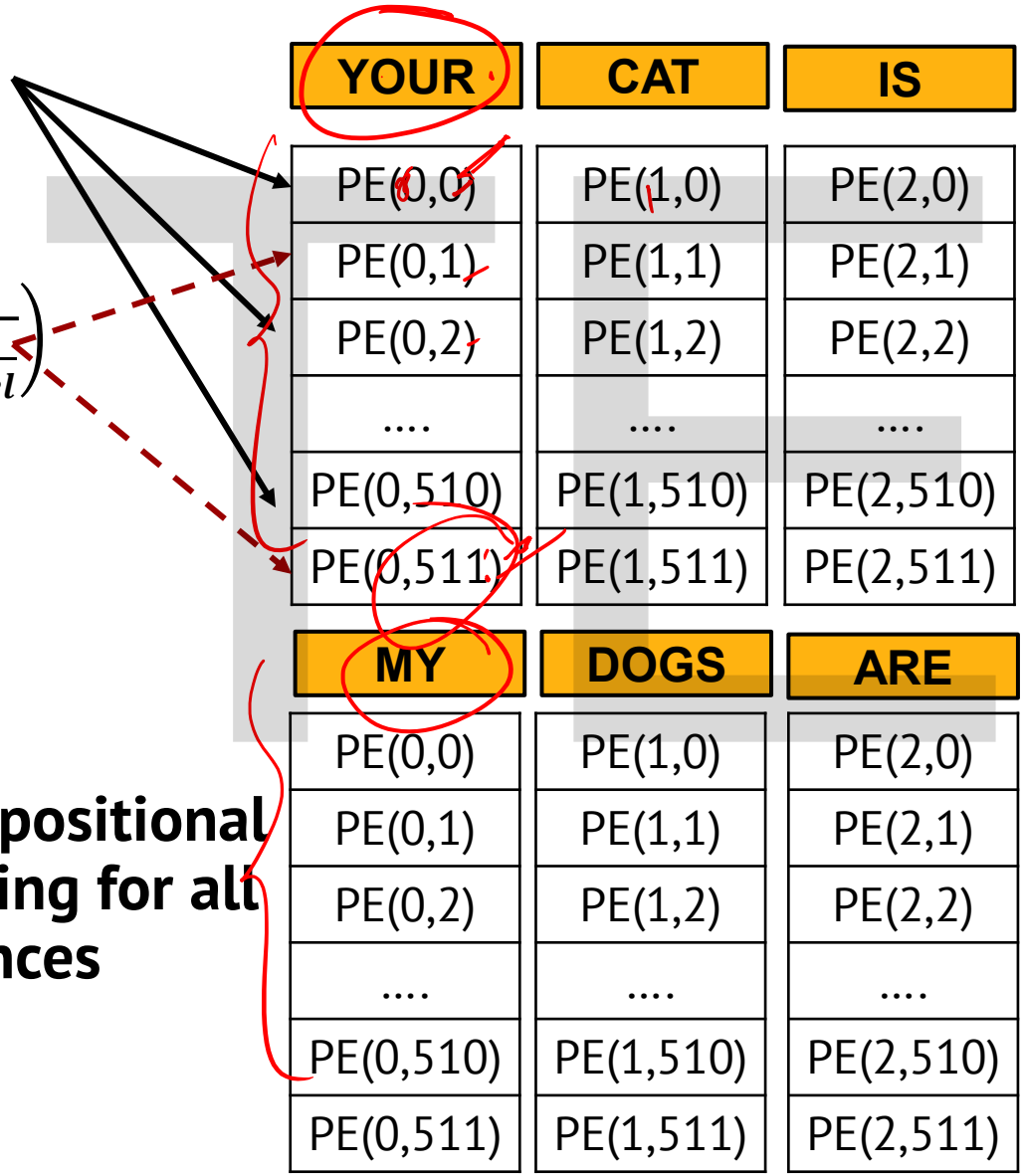
- Each word should carry some information about its sentence
  - Embedding does not carry any info about position
- Solution: Let the model treat words that appear close to each other as 'close; and words that are distant as 'distant'
- Positional embedding provides a pattern that a model can learn
- Positional encoding vector is not learnt
  - Solely depends on the position of the word



NPTEL

Positional Encoding (How are they Calculated?)

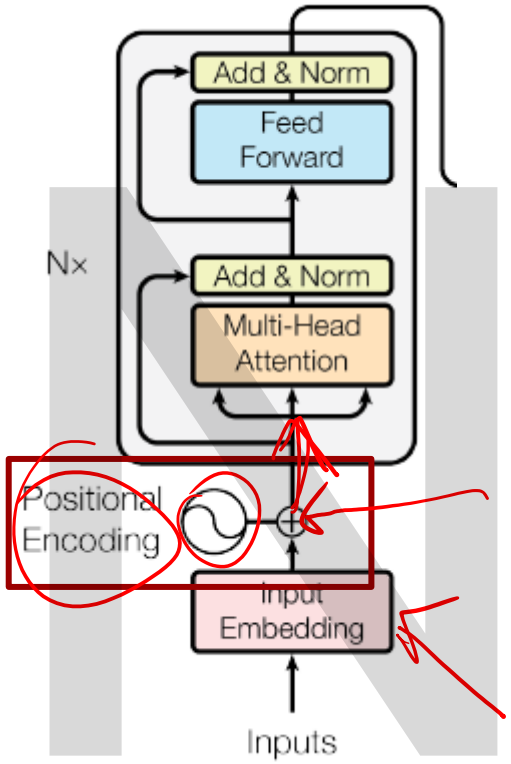
- $PE(pos, 2i) = sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$
  - for even dimension
- $PE(pos, 2i+1) = cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$
  - For odd dimension



Same positional encoding for all sentences

NPTEL

Encoder: Positional Encoding



Original sentence (tokens)

Embedding (vector of size 512)
d_{model}=512

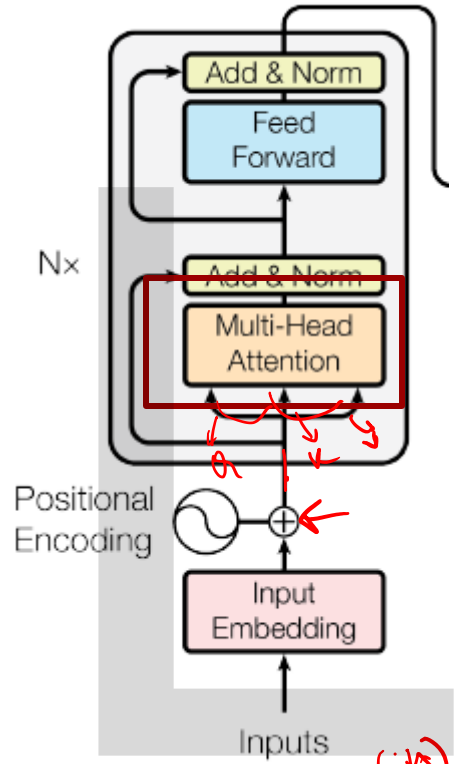
Position Embedding (vector of size 512)
Computed Once and Reused for every sentence

Input to the Encoder
(vector of size 512)



Encoder: Self-Attention

- Was introduced before introduction of transformer
  - Multi-headed attention was introduced
- $Attention\,(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- Self-attention allows the model to relate words to each other
- Consider sequence length seq=6 and d_{model}=512
- Q, K, V are the input sequence



Q = K = V = Emb + Enc(i/p)

NPTEL

Encoder: Self-Attention

$\boldsymbol{Attention}\,(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \boldsymbol{softmax}\left(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}}\right)\boldsymbol{V}$; Represents a score of how one word is related to another

The matrices **Q**, **K** and **V** are just the input sentence.

$$\text{softmax}\left(\frac{\mathbf{Q}_{(6,512)} \times \mathbf{K}^T_{(512,6)}}{\sqrt{512}}\right) =$$

	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

(6, 6)

* for simplicity I considered only one head, which makes $d_{model} = d_k$.

Encoder: Self-Attention

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

(6, 6)

X

V

(6, 512)

=

Attention

(6, 512)

Each row in this matrix captures not only the meaning (given by the embedding) or the position in the sentence (represented by position encoding) but also each word's interaction with other words

Properties of Self-Attention

- Self-Attention is permutation invariant
  - Shuffling the words will not change the value; just change the position
- Self-attention does not require any parameter
- Values along the diagonal are the highest
- If we don't want some position to interact, we can always set their values as $-\infty$
  $-e^{-\infty} = 0$

NPTEL

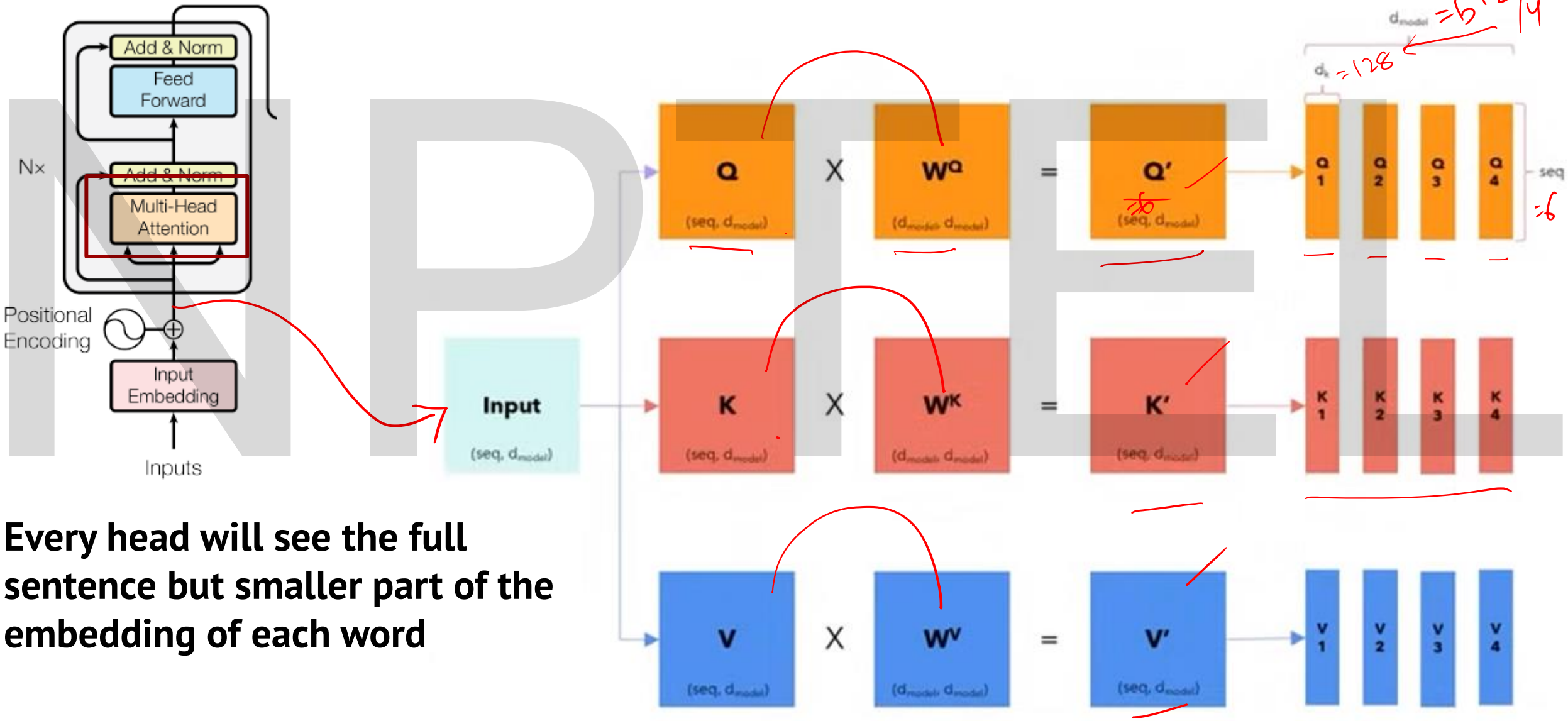
Multi-head Attention

- $\boldsymbol{Attention\,(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V}$
- $\boldsymbol{Multihead\,(Q,K,V) = Concat(head_1, \ldots . head_h)W^o}$

  $\boldsymbol{-head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)}$

NPTEL

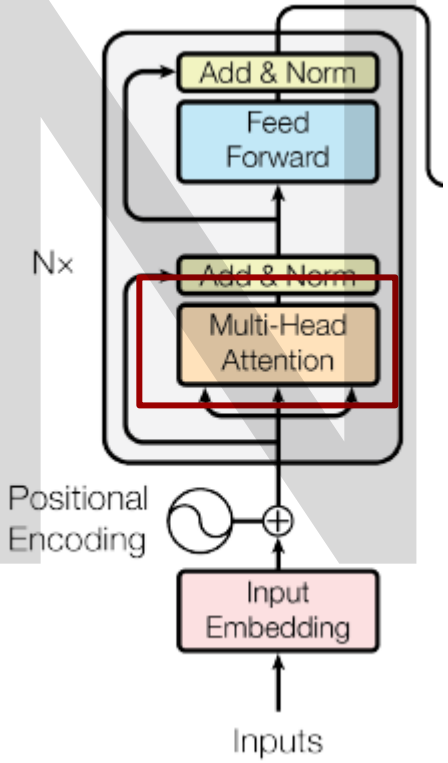
Multi-head Attention

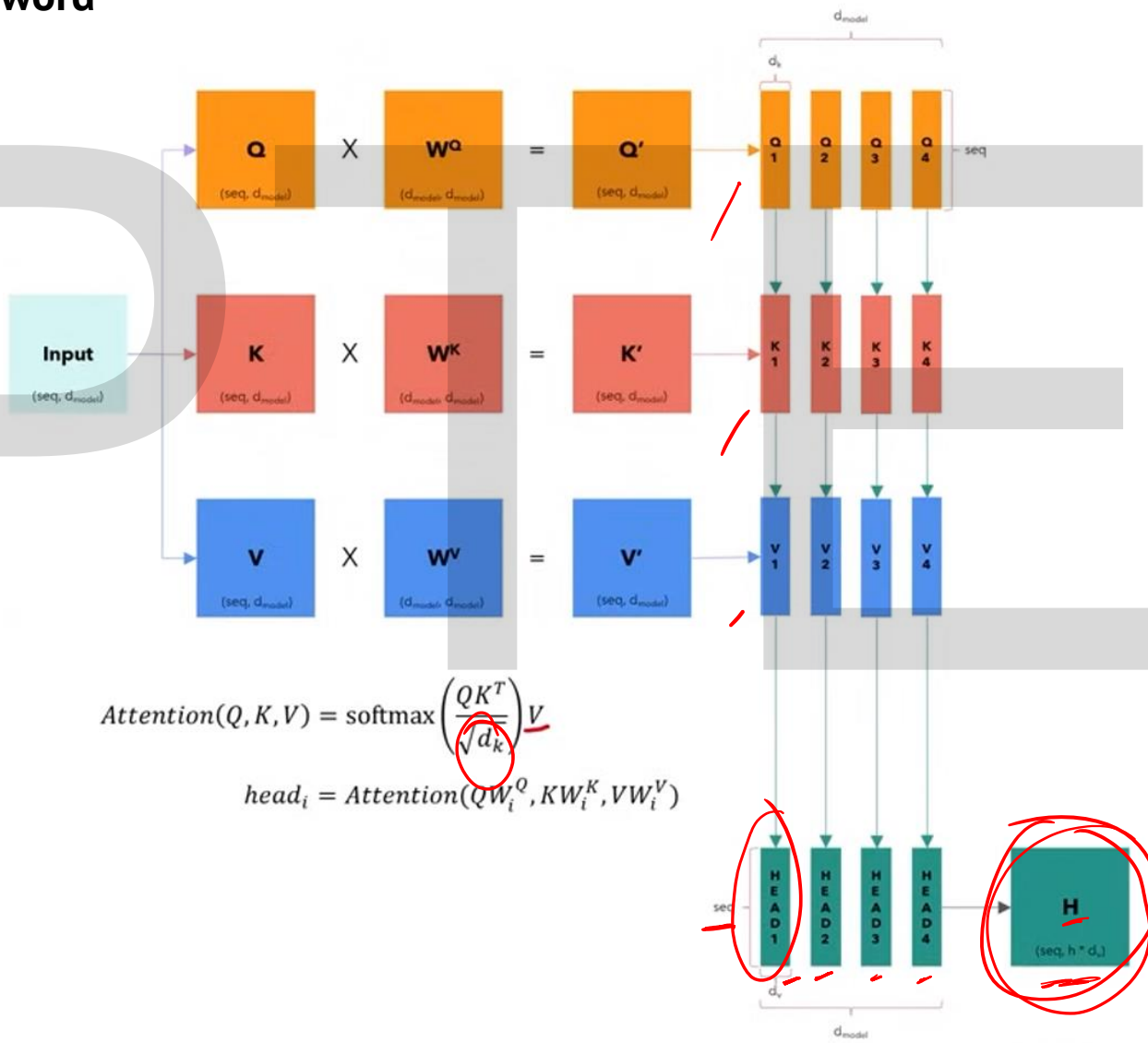


Every head will see the full sentence but smaller part of the embedding of each word

Multi-head Attention

Each head learns different aspect of a word



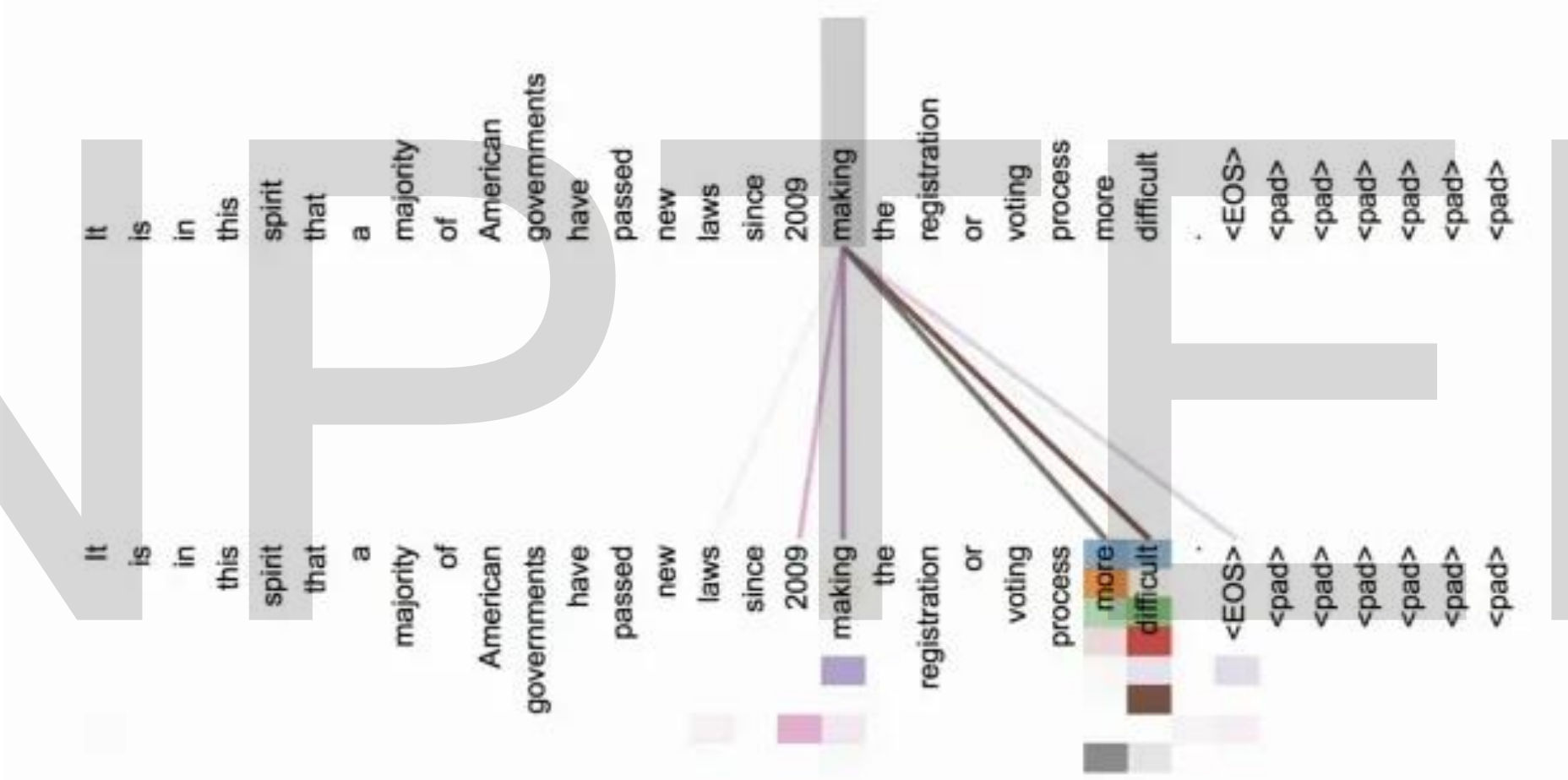


$$Attention(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

$$MultiHead(Q,K,V) = Concat(head_1 \dots head_h)W^O$$

Visualizing Attentions

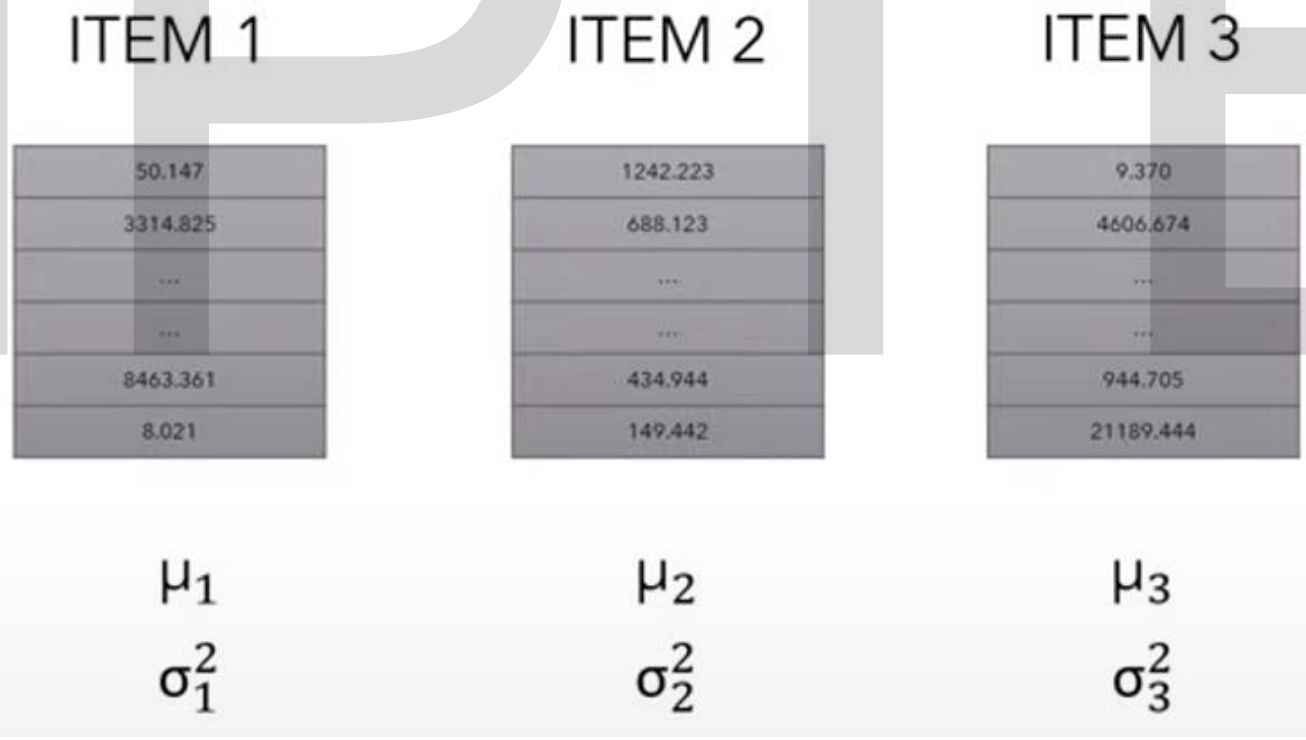


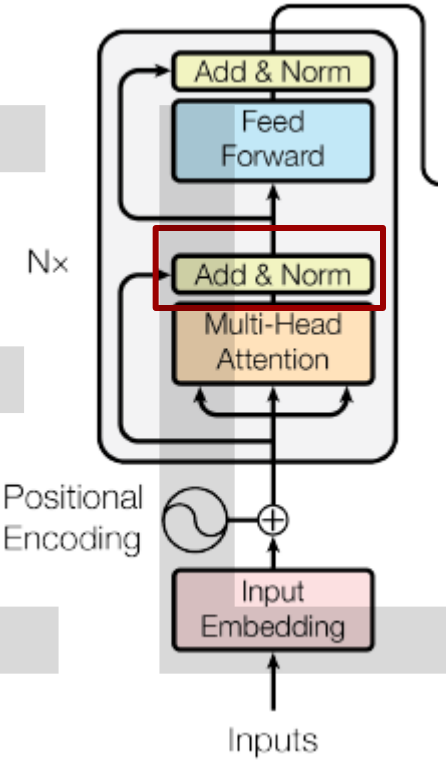
NPTEL

Encoder: Add and Norm

- **Similar to batch normalization in DNN**

$$-\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

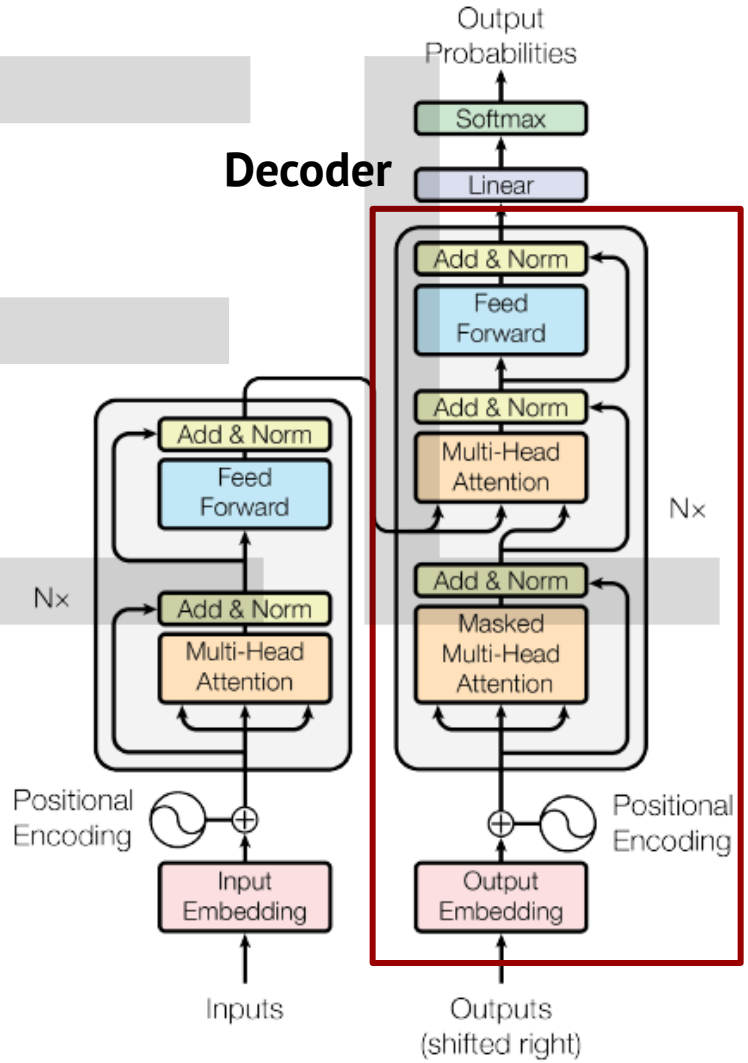




NPTEL

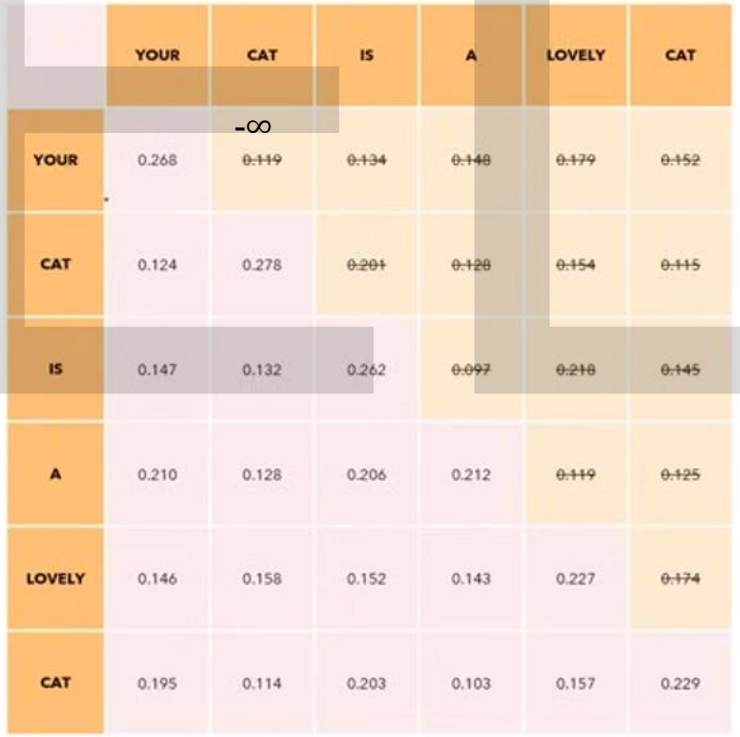
Decoder

- Output embedding is similar to the input embedding
- Masked Multi-head Attention is same as self-attention on Decoder inputs
- Multi-head Attention
  - Keys (K) and Values (V) are coming from Encoder
  - Queries (Q) are coming from itself
  - No longer self-attention
  - Cross-attention
- Feed forward is a fully connected layer



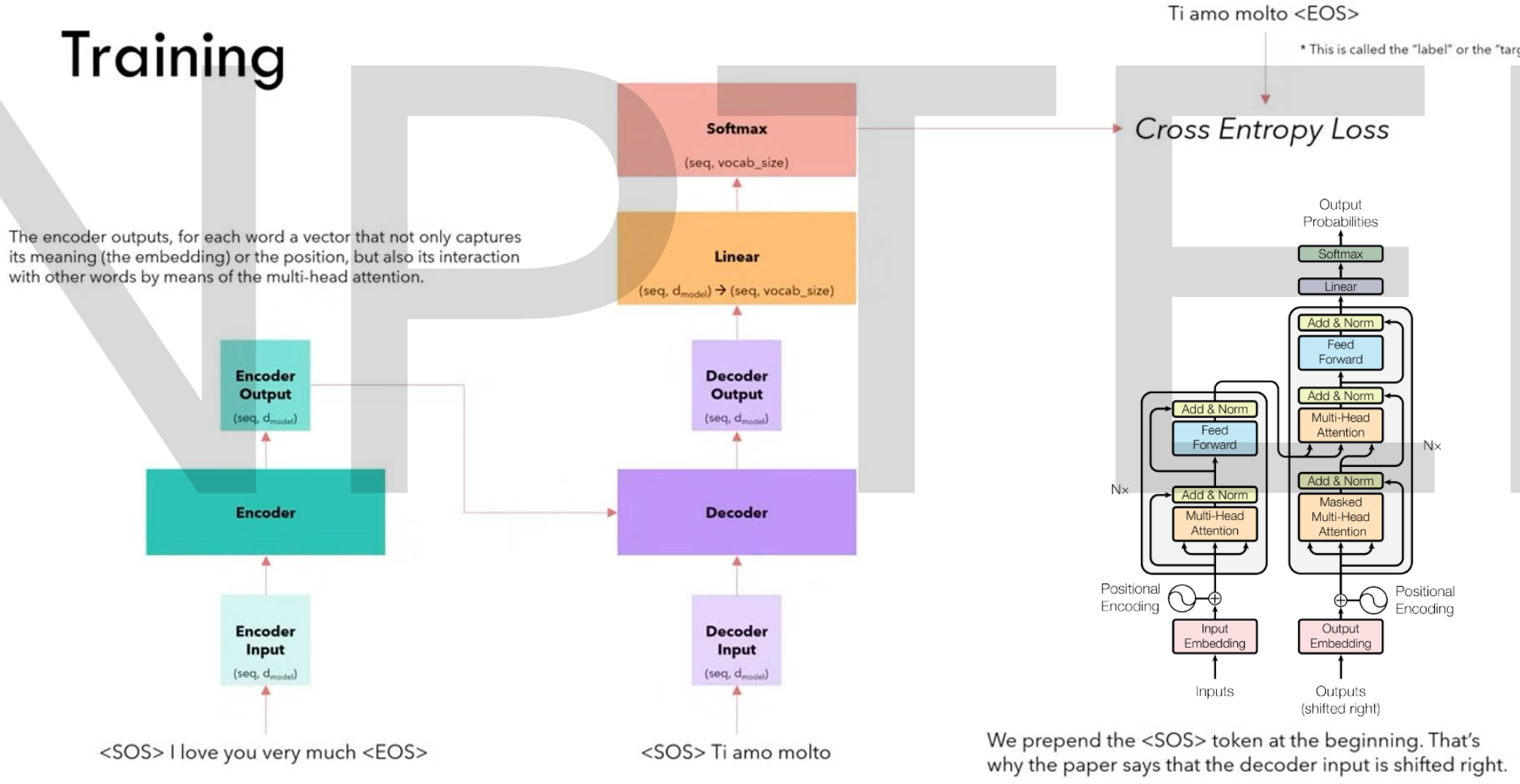
Decoder: Masked Multi-Headed Attention

- Goal is to make the model causal
  - Output at a certain position can only depend on the words on the previous positions
  - The model must not see future words
- The values are made $-\infty$
  - Softmax values are zero



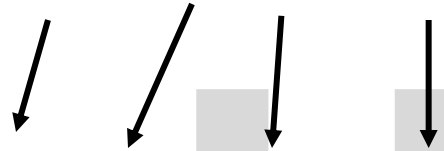
NPTEL

Training of a Transformer Model



Training of a Transformer Model

- <SOS> Ti amo molto



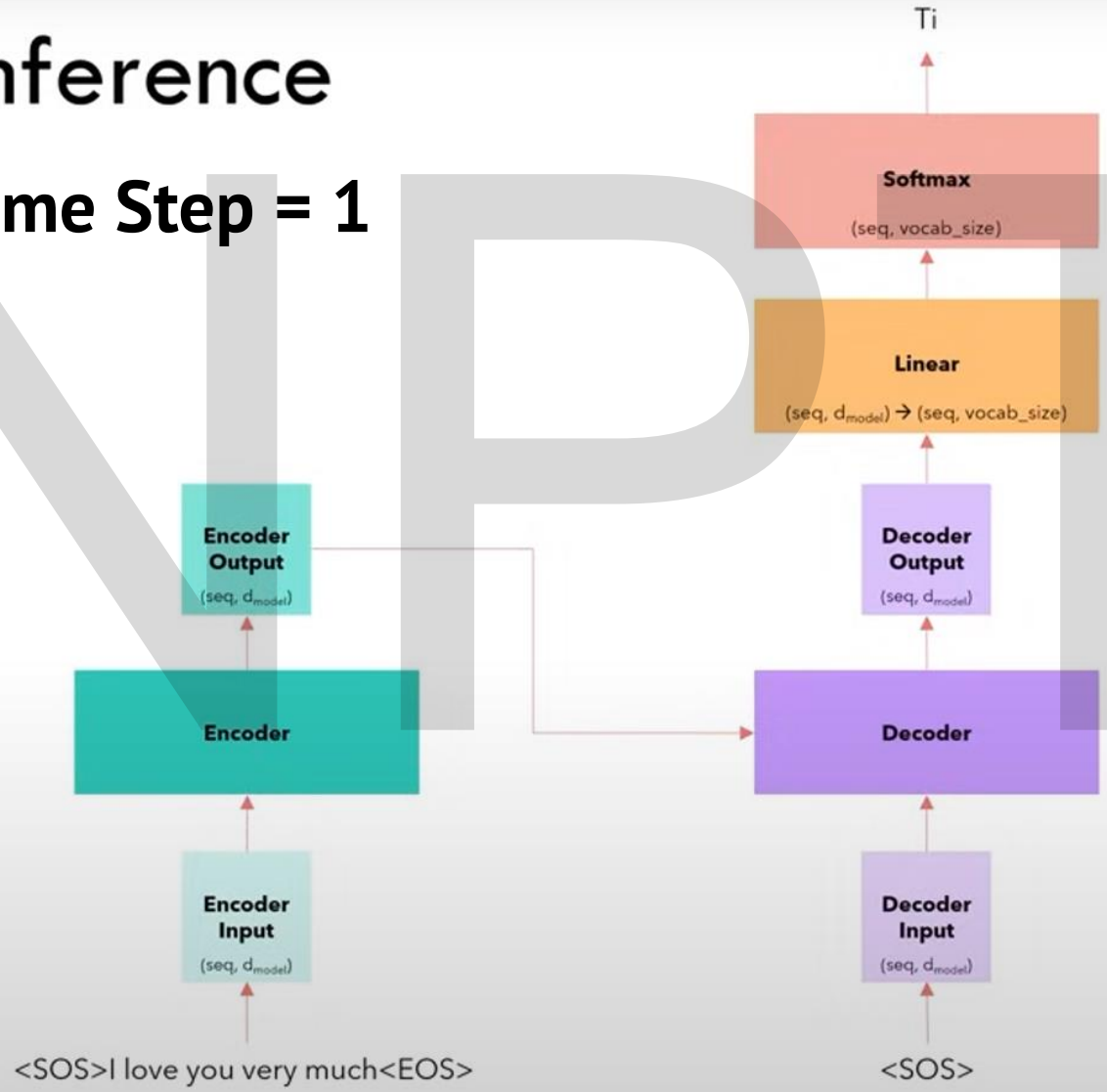
- Ti amo molto <EOS>
- It all happens in one time step
  - Unlike RNNs

NPTEL

Inference

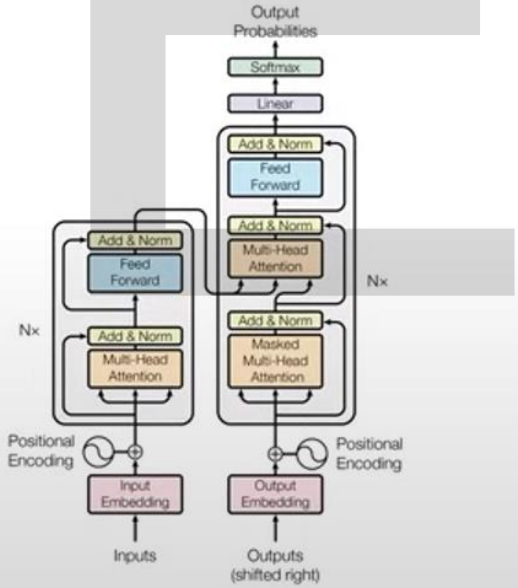
Inference

Time Step = 1



We select a token from the vocabulary corresponding to the position of the token with the maximum value.

The output of the last layer is commonly known as **logits**



* Both sequences will have same length thanks to padding

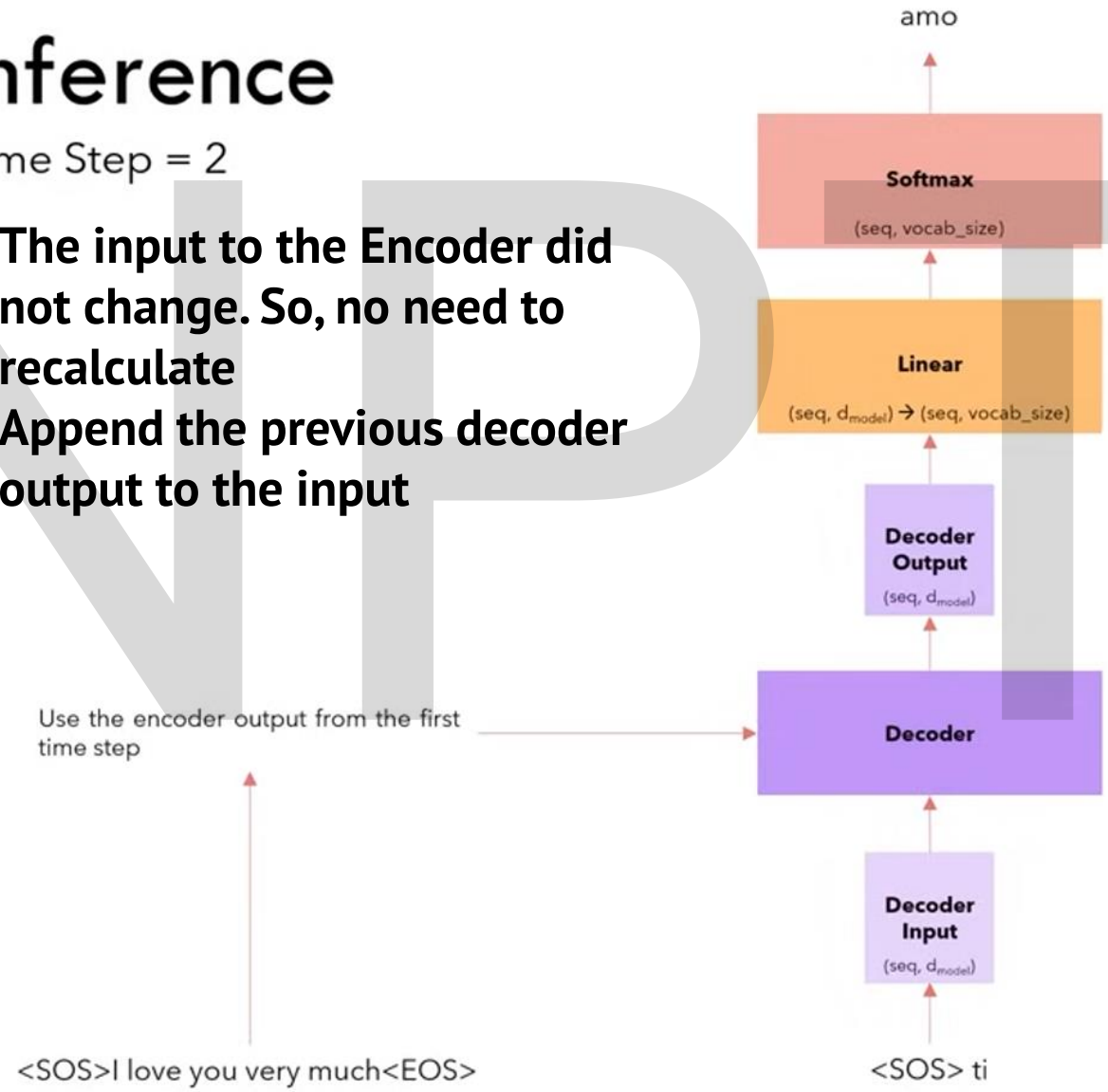
NPTEL

Inference

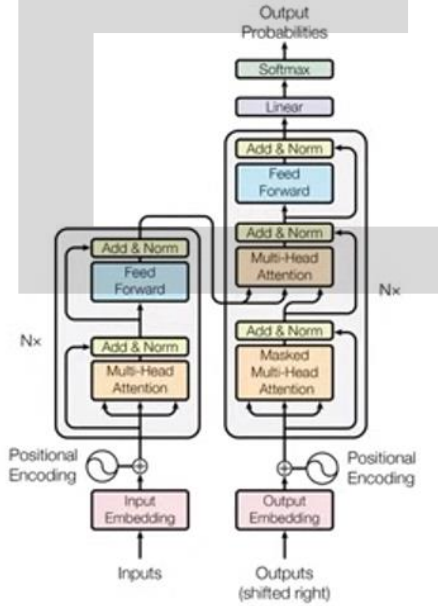
Inference

Time Step = 2

- **The input to the Encoder did not change. So, no need to recalculate**
- **Append the previous decoder output to the input**



Since decoder input now contains **two** tokens, we select the softmax corresponding to the second token.

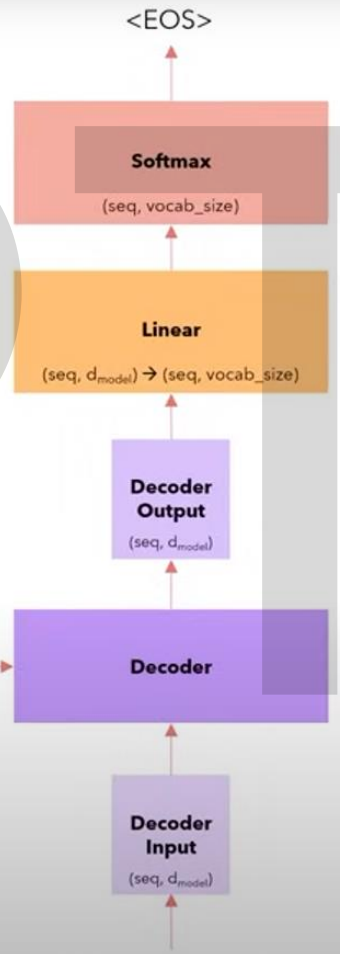


Append the previously output word to the decoder input

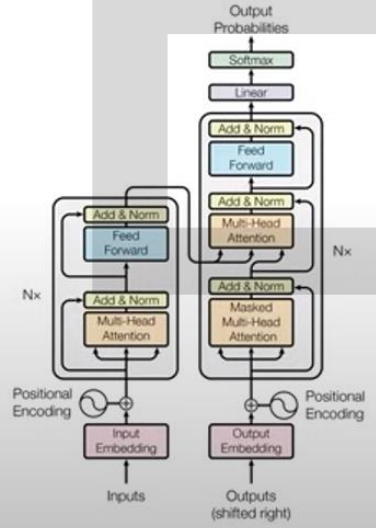
Inference

Inference

Time Step = 4



Since decoder input now contains **four** tokens, we select the softmax corresponding to the fourth token.



Use the encoder output from the first time step

<SOS>I love you very much<EOS>

<SOS> ti amo molto

Append the previously output word to the decoder input



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)